Hooks

Hooks allow you to declare chunks of code that run before, after, or around your specs. A hook can run for each :example, once for each :context, or globally for the entire :suite.

We looked at hooks in detail in *Hooks*, on page 113. As a reminder, here's a typical before hook defined in an RSpec.configure block:

09-configuring-rspec/03/rspec_configure.rb

```
RSpec.configure do |config|
  config.before(:example) do
    # ...
  end
end
```

We'd like to review one other special-purpose configuration hook that doesn't fit the typical before/after/around pattern. In *Isolating Your Specs Using Database Transactions*, on page 92, you saw a way to run expensive database setup code on demand, the first time it's needed:

09-configuring-rspec/03/rspec_configure.rb

```
RSpec.configure do |config|
  config.when_first_matching_example_defined(:db) do
    require 'support/db'
  end
end
```

This hook uses metadata (the :db symbol) to perform extra configuration just for the specs that need it.

While config hooks are a great way to reduce duplication and keep your examples focused, there are significant downsides if you overuse them:

- A slow test suite due to extra logic running for every example
- Specs that are harder to understand because their logic is hidden in hooks

To avoid these pitfalls while keeping your specs organized, you can use a simpler, more explicit technique: using Ruby modules inside your configure blocks.

Sharing Code Using Modules

Modules are one of Ruby's main tools for sharing code. You can add all of a module's methods into a class by calling include or prepend:

09-configuring-rspec/03/rspec_configure.rb

```
class Performer
  include Singing # won't override Performer methods
  prepend Dancing # may override Performer methods
end
```